

PROJECT: Scalable Microservices Architecture with ECS and Fargate

Services: ECS, Fargate, ECR, RDS, CloudWatch

Description: Containerize a microservices-based application using Docker. Deploy using ECS Fargate and manage logs with CloudWatch. Use RDS as the backend database.

Skills: Containers, microservices, monitoring

Definition: A Scalable Microservices Architecture with ECS and Fargate is a cloud-based system design approach where an application is broken into small, independent services (microservices) that can be developed, deployed, and scaled separately. These services are typically containerized using Docker and managed using Amazon Elastic Container Service (ECS), while AWS Fargate is used to run those containers without managing servers (serverless containers). Container images are stored in Amazon Elastic Container Registry (ECR), and data is often managed using Amazon RDS

Scope of the project:

The scope of this project focuses on building and deploying a scalable microservice architecture in the cloud using AWS services. It covers containerization of applications, storing images in ECR, and deploying containers using ECS Fargate. The project also includes monitoring application logs through CloudWatch for troubleshooting and performance monitoring. This approach can be extended to deploy multiple microservices and integrate additional services such as databases, load balancers, and CI/CD pipelines.

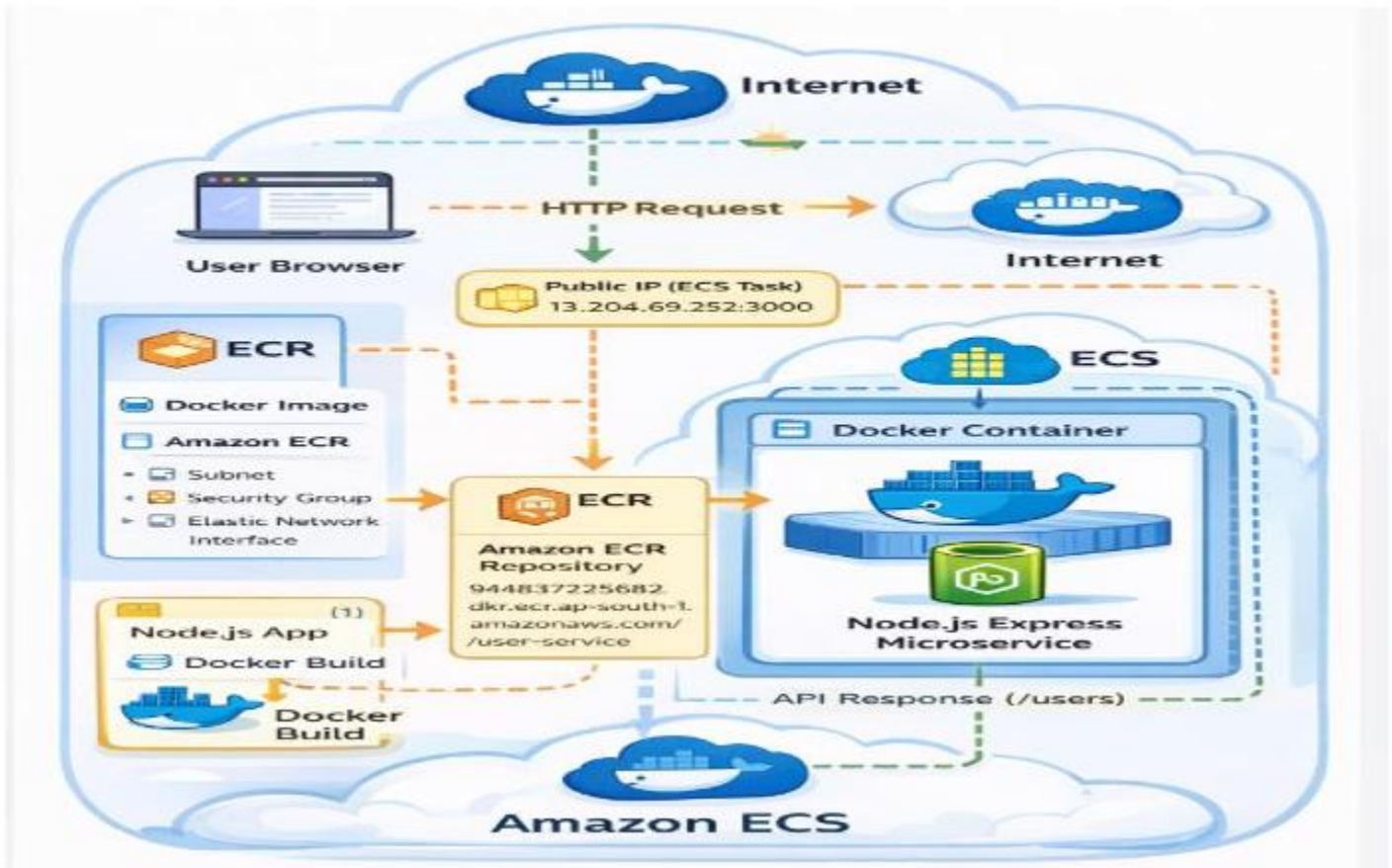
Methodologies:

1. Design the application as independent microservices (each service handles one function).
2. Containerize each microservice using Docker.
3. Push Docker images to Amazon Elastic Container Registry (ECR).
4. Create task definitions for each service in Amazon Elastic Container Service (ECS).
5. Deploy containers using AWS Fargate (serverless compute).
6. Configure networking using VPC, subnets, and security groups.
7. Connect microservices to database using Amazon RDS.
8. Set up load balancing using Application Load Balancer (ALB).
9. Enable logging and monitoring with Amazon CloudWatch.
10. Implement auto scaling based on CPU/memory usage.
11. Continuously integrate and deploy using CI/CD pipeline (optional Jenkins/GitHub Actions).
12. Test each microservice independently and as a full system.

Services Used in this Project:

1. Amazon Elastic Container Service (ECS) – to deploy and manage microservices containers
2. AWS Fargate – to run containers without managing servers
3. Amazon Elastic Container Registry (ECR) – to store Docker images
4. Amazon RDS – to manage relational database backend
5. Amazon CloudWatch – to monitor logs and metrics

Architecture Diagram:



Implementation and Execution of Scalable Microservices Architecture with ECS and Fargate.

STEP 1: Launch an instance

The screenshot shows the AWS Management Console for the EC2 service. The left sidebar contains navigation links for EC2, Dashboard, AWS Global View, Events, Instances, Instance Types, and Launch Templates. The main content area displays the 'Instances (1/1)' page. At the top, there are buttons for 'Connect', 'Instance state', 'Actions', and 'Launch instances'. Below these is a search bar and a table of instances. The table has columns for Name, Instance ID, Instance state, Instance type, Status check, Alarm status, and Availability. One instance, 'scalable-microservice', is listed with Instance ID 'i-09eb8c8b9465dbddd', in a 'Running' state, of type 't3.micro', with '3/3 checks passed', and in the 'ap-south' region.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
scalable-microservice	i-09eb8c8b9465dbddd	Running	t3.micro	3/3 checks passed	View alarms +	ap-south

STEP 2: Update the Server and Install Docker and verified Docker

The first screenshot shows a terminal window on an Amazon Linux 2023 instance. The user is logged in as 'ec2-user' and has run 'sudo su' to become root. They then execute 'yum update -y', which updates the system. The output shows the last login time, the command being run, and the result of the update: 'Dependencies resolved. Nothing to do. Complete!'. The second screenshot shows the same terminal window after the user has run 'docker --version'. The output displays 'Docker version 25.0.14, build 0bab007'.

```
#
#####
~\#####\
~~\###|
~~\##/
~~\V~'~>
~~~
~~~
~~~
Last login: Fri Apr 17 11:46:00 2026 from 13.233.177.3
[ec2-user@ip-172-31-120-88 ~]$ sudo su
[root@ip-172-31-120-88 ec2-user]# yum update -y
Last metadata expiration check: 9:34:02 ago on Fri Apr 17 03:00:35 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-120-88 ec2-user]#
```

```
#
#####
~\#####\
~~\###|
~~\##/
~~\V~'~>
~~~
~~~
~~~
Last login: Fri Apr 17 11:46:00 2026 from 13.233.177.3
[ec2-user@ip-172-31-120-88 ~]$ sudo su
[root@ip-172-31-120-88 ec2-user]# yum update -y
Last metadata expiration check: 9:34:02 ago on Fri Apr 17 03:00:35 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-120-88 ec2-user]# ^C
[root@ip-172-31-120-88 ec2-user]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-120-88 ec2-user]#
```

STEP 3: Create a directory name Microservices Project

The screenshot shows a terminal window where the user has created a directory named 'microservices-project' and then listed its contents. The output shows the directory 'microservices-project' and its subdirectories 'app.js', 'order-service', 'product-service', and 'user-service'.

```
[root@ip-172-31-120-88 ec2-user]# ls
microservices-project
[root@ip-172-31-120-88 ec2-user]# cd microservices-project/
[root@ip-172-31-120-88 microservices-project]# ls
app.js  order-service  product-service  user-service
```

STEP 4: Create application file

```
[root@ip-172-31-120-88 microservices-project]# cd user-service/  
[root@ip-172-31-120-88 user-service]# nano app.js  
[root@ip-172-31-120-88 user-service]#
```

```
GNU nano 8.3 app.js  
const express = require('express');  
const app = express();  
  
app.get('/', (req, res) => {  
  res.send('Hello from Microservice!');  
});  
  
app.listen(3000, '0.0.0.0', () => {  
  console.log('Server running on port 3000');  
});
```

STEP 6: Install Dependencies

```
aws [Alt+S]  
[ec2-user@ip-10-0-2-213 user-service]$ npm init -y  
Wrote to /home/ec2-user/microservices-project/user-service/package.json:  
  
{  
  "name": "user-service",  
  "version": "1.0.0",  
  "main": "app.js",  
  "scripts": {  
    "test": "echo \\\"Error: no test specified\\\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "description": ""  
}  
  
[ec2-user@ip-10-0-2-213 user-service]$
```

```
[ec2-user@ip-10-0-2-213 user-service]$ npm install express  
  
added 65 packages, and audited 66 packages in 3s  
  
22 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities  
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 7: Create Dockerfile

```
[root@ip-172-31-120-88 user-service]# nano Dockerfile
[root@ip-172-31-120-88 user-service]#
```

```
GNU nano 8.3 Dockerfile
FROM node:18

WORKDIR /app

COPY . /app

RUN npm install

CMD ["node", "./app.js"]
```

STEP 8: Build Docker Image

```
[root@ip-172-31-120-88 user-service]# docker build -t user-service .
[+] Building 0.8s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 179B
=> [internal] load metadata for docker.io/library/node:18
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783
=> [internal] load build context
=> => transferring context: 81.20kB
=> CACHED [2/4] WORKDIR /app
=> CACHED [3/4] COPY . /app
=> CACHED [4/4] RUN npm install
=> exporting to image
=> => exporting layers
=> => writing image sha256:cb128d4826d9c45a8819d0dcd1b7ed6d8579fb548a3ca33171a1ea4b8079b956
=> => naming to docker.io/library/user-service
[root@ip-172-31-120-88 user-service]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
user-service	latest	cb128d4826d9	2 minutes ago	1.09GB
074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo	latest	573674bd266e	About an hour ago	1.09GB
<none>	<none>	4947cbfecdd55	About an hour ago	1.09GB

```
[root@ip-172-31-120-88 user-service]#
```

STEP 9: docker ps

```
[root@ip-172-31-120-88 user-service]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
659429fe3964	573674bd266e	"docker-entrypoint.s..."	About an hour ago	Up About an hour	0.0.0.0:3001->3000/tcp, :::3001->3000/tcp	cranky_ne...
5741c08149a6	573674bd266e	"docker-entrypoint.s..."	About an hour ago	Up About an hour	0.0.0.0:3000->3000/tcp, :::3000->3000/tcp	nifty_ja...

```
[root@ip-172-31-120-88 user-service]#
```

STEP 10: Login Docker to ECR

```
[ec2-user@ip-172-31-120-88 ~]$ aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-
stdin 123456789012.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

STEP 11: Tagged and pushed the Docker image

```
Login Succeeded
[ec2-user@ip-172-31-120-88 ~]$ ^C
[ec2-user@ip-172-31-120-88 ~]$ docker tag user-service:latest 074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo:latest
[ec2-user@ip-172-31-120-88 ~]$ docker push 074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo
Using default tag: latest
The push refers to repository [074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo]
```

STEP 12: Create ECS Cluster

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions New

Account settings

AWS Batch

Introducing managed daemons

You can now manage software agents as standalone daemons across container instances in a capacity provider, independent of your applications. [Learn more](#)

Create daemon task definition

Clusters (1)

Search clusters

By default, we only load up to 1,000 clusters at a time.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring
micro-cluster1	1	0 Pending 1 Run...	0 EC2	Default

STEP 13: Create Task Definition

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions New

Account settings

Task definitions (1)

Filter task definitions

Filter status: Active

Task definition	Status of last revision
micro-task	Active

STEP 14: Create ECS Service

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Daemon task definitions New

Account settings

AWS Batch

Amazon ECR

Repositories

micro-cluster1

Services

micro-service

Health

micro-service

Service overview

Status: Active

Tasks (1 Desired): 0 Pending | 1 Running

Task definition: revision micro-task:1

Deployment status: Success

Health and metrics

Status

Service name: micro-service

Service ARN: arn:aws:ecs:ap-south-1:074139044318:service/micro-cluster1/micro-service

Deployments current state: 1 Completed task

Created at: April 17, 2026, 16:21 (UTC+5:30)

STEP 15: Access the Service

User Service Running

STEP 16: Monitoring Application Logs using AWS CloudWatch

- View Log Streams

CloudWatch

Favorites and recents

Alarms

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

▼ Logs

Log Management

Log Analytics Preview

Log Anomalies

/ecs/micro-task

Actions

View in Logs Insights

Start tailing

Log group details

Log streams

Tags

Data protection

Anomaly detection

Metric filters

Subscription filter

Log streams (17)

By default, we only load the most recent log streams.

Filter log streams or try prefix search

Exact match

Show expired

Log stream	Last event time
ecs/micro-container/e618f4db1ce344a8ba055a24c087d10c	2026-04-17 17:21:04 (UTC+05:30)
ecs/micro-container/f1050b1a593d4ae5bd18a288561bf2f0	2026-04-17 17:12:07 (UTC+05:30)

- View Log Events

CloudWatch

Favorites and recents

Alarms

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

Log events

Actions

Start tailing

You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns

Filter events - press enter to search

1m

1h

Local

Display

Timestamp	Message
No older events at this moment. Retry	
2026-04-17T17:21:04.415+05:30	Server running on port 3000

Troubleshooting:

- **Issue:** Docker images could not be pushed to Amazon ECR due to authentication errors.
- **Solution:** AWS CLI authentication was configured using the `aws ecr get-login-password` command to securely log in to the ECR repository before pushing the image.

Conclusion:

In conclusion, the Scalable Microservices Architecture using ECS and Fargate provides an easy and efficient way to deploy containerized applications. It allows services to run independently and scale based on demand without managing servers. With services like ECR for images, RDS for databases, and CloudWatch for monitoring, the system becomes more reliable and manageable. This approach improves scalability, performance, and simplifies overall application deployment in the cloud.